Big Data Infrastructure

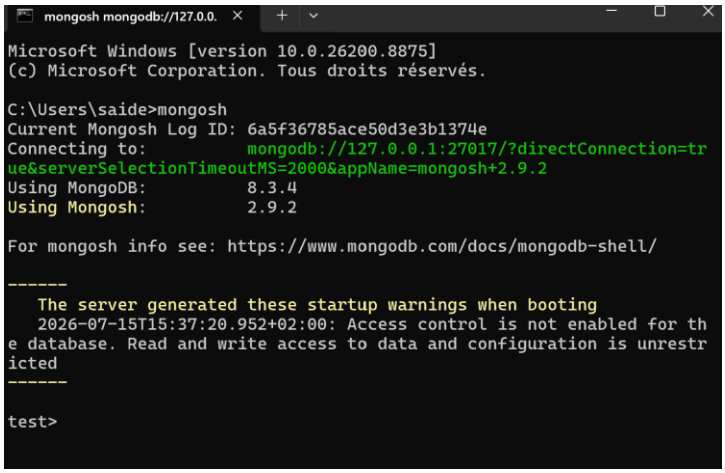
Le manuel de MongoDB : https://docs.mongodb.com/manual/

I. Interrogation d'une base de documents avec MongoDB

3. Depuis le conteneur my-mongo , retrouver le même fichier dans /mnt/ et importer le dans la base de donnée tp2-db au sein de la collection restaurants avec

mongoimport --db tp2-db --collection restaurants --drop --file /mnt/primer-dataset.json

Sur l'invite de la commande j'ai lancé MongoDB à l'aide de mongosh.



mongoimport –version

```
C:\Users\saide>mongoimport --version
mongoimport version: 100.17.0
git version: 0b142f65e139525881b6a343bc56d8d98e5a1f90
Go version: go1.25.9
   os: windows
   arch: amd64
   compiler: gc
```

Si c'est bon (sinon télécharger le paquet sur le site MongoDB)

Adresse 61 rue sifax Orly 94310 – France Tél. : +33 (L.G.) / Fax :

www.sazotech-solutions.fr// info@sazotech.fr

Après :
mongoimport --db tp2-db --collection restaurants --drop –file
"C:\Users\saide\Desktop\ISOSET\TP\mongoDB\primer-dataset.json"

```
C:\Users\saide>mongoimport --db tp2-db --collection restaurants --drop --file "C:\Users\saide\Desktop\ISOSET\TP\mongoDB\
primer-dataset.json"
2026-07-21T11:27:07.633+0200    connected to: mongodb://localhost/
2026-07-21T11:27:07.709+0200    dropping: `tp2-db.restaurants`
2026-07-21T11:27:08.780+0200    25359 document(s) imported successfully. 0 document(s) failed to import.
```

4. Connectez-vous à la base tp2-db et répondez aux questions suivantes

Ensuite mongosh (dans un terminal, séparément)

(a) Afficher la liste de tous les documents de la collection restaurants

```
test> use tp2-db
switched to db tp2-db
tp2-db> db.restaurants.countDocuments()
25359
tp2-db>
```

```
tp2-db> db.restaurants.find()
[
  {
    _id: ObjectId('6a5f3b6bcef46c8618b074b0'),
    address: {
      building: '469',
      coord: [ -73.961704, 40.662942 ],
      street: 'Flatbush Avenue',
      zipcode: '11225'
    },
    borough: 'Brooklyn',
```


Adresse 61 rue sifax Orly 94310 – France Tél. : +33 (L.G.) / Fax :

www.sazotech-solutions.fr// info@sazotech.fr

(b) Donner la liste des restaurants situés dans le quartier (“borough”) Bronx

```
tp2-db>

tp2-db> db.restaurants.find({ borough: "Bronx" })
[
  {
    _id: ObjectId('6a5f3b6bcef46c8618b074b1'),
    address: {
      building: '1007',
      coord: [ -73.856077, 40.848447 ],
      street: 'Morris Park Ave',
      zipcode: '10462'
    },
    borough: 'Bronx',
    cuisine: 'Bakery',
    grades: [
      { date: ISODate('2014-03-03T00:00:00.000Z'), grade: 'A', score: 2 },
      { date: ISODate('2013-09-11T00:00:00.000Z'), grade: 'A', score: 6 },
      { date: ISODate('2013-01-24T00:00:00.000Z'), grade: 'A', score: 10 },
      { date: ISODate('2011-11-23T00:00:00.000Z'), grade: 'A', score: 9 },
      { date: ISODate('2011-03-10T00:00:00.000Z'), grade: 'B', score: 14 }
    ],
    name: 'Morris Park Bake Shop',
    restaurant_id: '30075445'
  },
```

(c) Donner la liste des restaurants qui proposent de la cuisine italienne

```
]
tp2-db> db.restaurants.find({ cuisine: "Italian" }).limit(1)
[
  {
    _id: ObjectId('6a5f3b6bcef46c8618b074e4'),
    address: {
      building: '10004',
      coord: [ -74.03400479999999, 40.6127077 ],
      street: '4 Avenue',
      zipcode: '11209'
    },
    borough: 'Brooklyn',
    cuisine: 'Italian',
    grades: [
      { date: ISODate('2014-02-25T00:00:00.000Z'), grade: 'A', score: 12 },
      { date: ISODate('2013-06-27T00:00:00.000Z'), grade: 'A', score: 7 },
      { date: ISODate('2012-12-03T00:00:00.000Z'), grade: 'A', score: 10 },
      { date: ISODate('2011-11-09T00:00:00.000Z'), grade: 'A', score: 12 }
    ],
    name: 'Philadelhia Grille Express',
    restaurant_id: '40364305'
  }
]
tp2-db>
```


Adresse 61 rue sifax Orly 94310 – France Tél. : +33 (L.G.) / Fax :

www.sazotech-solutions.fr// info@sazotech.fr

(d) Donner la liste des restaurants situés dans Manhattan et qui proposent de la cuisine irlandaise ("Irish")

```
tp2-db> db.restaurants.find({ borough: "Manhattan", cuisine: "Irish" }).limit(1)
[
  {
    _id: ObjectId('6a5f3b6bcef46c8618b074b4'),
    address: {
      building: '351',
      coord: [ -73.98513559999999, 40.7676919 ],
      street: 'West   57 Street',
      zipcode: '10019'
    },
    borough: 'Manhattan',
    cuisine: 'Irish',
    grades: [
      { date: ISODate('2014-09-06T00:00:00.000Z'), grade: 'A', score: 2 },
      { date: ISODate('2013-07-22T00:00:00.000Z'), grade: 'A', score: 11 },
      { date: ISODate('2012-07-31T00:00:00.000Z'), grade: 'A', score: 12 },
      { date: ISODate('2011-12-29T00:00:00.000Z'), grade: 'A', score: 12 }
    ],
    name: 'Dj Reynolds Pub And Restaurant',
    restaurant_id: '30191841'
  }
]
tp2-db>
```

(e) Donner la liste des restaurants qui proposent de la cuisine irlandaise ou italienne

```
tp2-db> db.restaurants.find({ cuisine: { $in: ["Irish", "Italian"] } }).limit(1)
[
  {
    _id: ObjectId('6a5f3b6bcef46c8618b074b4'),
    address: {
      building: '351',
      coord: [ -73.98513559999999, 40.7676919 ],
      street: 'West   57 Street',
      zipcode: '10019'
```

(f) Donner la liste des restaurants dont le code postal est 11225

```
tp2-db> db.restaurants.find({ "address.zipcode": "11225" }).limit(1)
[
  {
    _id: ObjectId('6a5f3b6bcef46c8618b074b0'),
    address: {
```

(g) Donner les restaurants dont les coordonnées sont (-73.8786113, 40.8502883)

```
tp2-db>

tp2-db> db.restaurants.find({ "address.coord": [-73.8786113, 40.8502883] }).limit(1)
[
  {
    _id: ObjectId('6a5f3b6bcef46c8618b074b8'),
    address: {
      building: '2300',
      coord: [ -73.8786113, 40.8502883 ],
```


Adresse 61 rue sifax Orly 94310 – France Tél. : +33 (L.G.) / Fax :

www.sazotech-solutions.fr// info@sazotech.fr

(h) Donner une liste de 5 restaurants qui ont reçu une note B

```
]
tp2-db> db.restaurants.find({ "grades.grade": "B" }).limit(5)
[
  {
    _id: ObjectId('6a5f3b6bcef46c8618b074b0'),
    address: {
      building: '469',
      coord: [ -73.961704, 40.662942 ],
      street: 'Flatbush Avenue',
      zipcode: '11225'
```

(i) Donner les noms des restaurants qui ont reçu une note A

```
tp2-db> db.restaurants.find(
|   { "grades.grade": "A" },
|   { name: 1, _id: 0 }
| )
[
  { name: "Wendy'S" },
  { name: 'Morris Park Bake Shop' },
  { name: 'Riviera Caterer' },
  { name: 'Tov Kosher Kitchen' },
  { name: 'Dj Reynolds Pub And Restaurant' },
  { name: 'Brunos On The Boulevard' },
  { name: "Wilken'S Fine Food" },
  { name: 'Kosher Island' },
  { name: 'Wild Asia' },
  { name: 'Carvel Ice Cream' },
  { name: 'Carvel Ice Cream' },
  { name: 'Nordic Delicacies' },
  { name: 'Seuda Foods' },
  { name: 'Glorious Food' },
  { name: 'The Movable Feast' },
  { name: "Sal'S Deli" },
  { name: "Bully'S Deli" },
  { name: "Steve Chu'S Deli & Grocery" },
  { name: "Harriet'S Kitchen" },
  { name: 'P & S Deli Grocery' }
]
Type "it" for more
tp2-db>
```


Adresse 61 rue sifax Orly 94310 – France Tél. : +33 (L.G.) / Fax :

www.sazotech-solutions.fr// info@sazotech.fr

(j) Pour les restaurant qui ont reçu une note A, donner le nom du restaurant, la date de l'évaluation et le score correspondant

```
tp2-db>

tp2-db> db.restaurants.aggregate([
|   { $unwind: "$grades" },
|   { $match: { "grades.grade": "A" } },
|   { $project: { _id: 0, name: 1, date: "$grades.date", score: "$grades.score" } }
| ])
[
  {
    name: "Wendy'S",
    date: ISODate('2014-12-30T00:00:00.000Z'),
    score: 8
  },
  {
    name: "Wendy'S",
    date: ISODate('2013-04-30T00:00:00.000Z'),
    score: 12
  },
  {
```

(k) Donner les noms des restaurants qui ont une évaluation (grades) dont le score est supérieur à 25

```
tp2-db> db.restaurants.find(
|   { "grades.score": { $gt: 25 } },
|   { name: 1, _id: 0 }
| )
[
  { name: 'Brunos On The Boulevard' },
  { name: 'P & S Deli Grocery' },
  { name: 'May May Kitchen' },
  { name: "Lorenzo & Maria'S" },
  { name: 'Terminal Cafe/Yankee Clipper' },
  { name: 'B & M Hot Bagel & Grocery' },
  { name: 'Texas Rotisserie' },
  { name: 'Old Town Bar & Restaurant' },
  { name: 'Polish National Home' },
  { name: 'Gottscheer Hall' },
  { name: 'Hot Bagels' },
  { name: 'Nyac Main Dining Room' },
  { name: 'Tout Va Bien' },
  { name: 'Marchis Restaurant' },
  { name: 'Towne Cafe' },
  { name: 'Shell Lanes' },
  { name: 'Como Pizza' },
  { name: 'Yankee Tavern' },
  { name: "Reynold'S Bar" },
  { name: 'King Yum Restaurant' }
]
Type "it" for more
tp2-db>
```


Adresse 61 rue sifax Orly 94310 – France Tél. : +33 (L.G.) / Fax :

www.sazotech-solutions.fr// info@sazotech.fr

(l) Même question que précédemment mais en triant les résultats par ordre croissant des noms des Restaurants

```
tp2-db> db.restaurants.find(
|   { "grades.score": { $gt: 25 } },
|   { name: 1, _id: 0 }
| ).sort({ name: 1 })
[
  { name: '#1 Sabor Latino Restaurant' },
  { name: "''W'' Cafe" },
  { name: '1 Buen Sabor' },
  { name: '100 Fun' },
  { name: '102 Noodles Town Restaurant' },
  { name: '1020 Bar' },
  { name: '10Th Avenue Cookshop' },
  { name: '121 Fulton Street' },
  { name: '123 Nikko' },
  { name: '15 East Restaurant' },
  { name: '16 Handles' },
  { name: '169 Bar' },
  { name: '18 Chinese Cuisine' },
  { name: '1St Avenue Gourmet' },
  { name: '2 Bros Pizza' },
  { name: '200 Fifth Avenue Restaurant & Sports Bar' },
  { name: '2058 Deli And Grocery' },
  { name: '216 Good Joy Chinese Restaurant' },
  { name: '218 Restaurant' },
  { name: '22 Thai Cuisine' }
]
Type "it" for more
tp2-db>
```

(m) Donner les noms des restaurants qui ont des évaluations (grades) dont le score est compris entre 6 et 10

```
tp2-db> db.restaurants.find(
|   { grades: { $elemMatch: { score: { $gte: 6, $lte: 10 } } } },
|   { name: 1, _id: 0 }
| )
[
  { name: "Wendy'S" },
  { name: 'Morris Park Bake Shop' },
  { name: 'Riviera Caterer' },
  { name: 'Brunos On The Boulevard' },
  { name: "Wilken'S Fine Food" },
  { name: 'Kosher Island' },
  { name: 'Carvel Ice Cream' },
  { name: 'Nordic Delicacies' },
  { name: 'Seuda Foods' },
  { name: 'Glorious Food' },
  { name: "Sal'S Deli" },
  { name: "Bully'S Deli" },
  { name: "Steve Chu'S Deli & Grocery" },
  { name: "Harriet'S Kitchen" },
  { name: 'P & S Deli Grocery' },
  { name: 'Angelika Film Center' },
  { name: 'May May Kitchen' },
  { name: 'Downtown Deli' },
  { name: '1 East 66Th Street Kitchen' },
  { name: "Shashemene Int'L Restaura" }
]
Type "it" for more
tp2-db>
```


Adresse 61 rue sifax Orly 94310 – France Tél. : +33 (L.G.) / Fax :

www.sazotech-solutions.fr// info@sazotech.fr

(n) Donner le nombre de restaurants localisés au Bronx

```
tp2-db> db.restaurants.countDocuments({ borough: "Bronx" })
2338
tp2-db>
```

(o) Donner le nombre de restaurants localisés au Bronx et proposant une cuisine italienne

```
tp2-db> db.restaurants.countDocuments({ borough: "Bronx", cuisine: "Italian" })
52
tp2-db>
```

II. Modélisation et implémentation d'un base de données documentaires

1. Proposer une approche par références (bdref) et une approche par documents imbriqués pour l'implémentation de votre base de données (bdimb)

L'enseignant Ens02 (Barbara) : son nom de famille n'est pas écrit (NaN) dans le tableau, mais la question (f) parle de "Durand Barbara" ➔ Donc on va considérer Nom = Durant, Prénom = Barbara.

Cours Info-01 : discipline vide (NaN) dans le tableau mais forcément "Informatique" (numéro Info-01) ; cours Maths-01 : intitulé vide mais "Mathématiques" logiquement.

Approche bdref (par références)

Créer la base bdref

```
tp2-db> use bdref
switched to db bdref
bdref>
```

Collection students :

```
bdref> db.students.insertMany([
|   { _id: "001", nom: "Dupont", prenom: "Bob", tel: "0600210000" },
|   { _id: "002", nom: "Durant", prenom: "Paul", tel: null },
|   { _id: "003", nom: "Dupont", prenom: "Toto", tel: null }
| ])
|
{
  acknowledged: true,
  insertedIds: { '0': '001', '1': '002', '2': '003' }
}
bdref>
```


Adresse 61 rue sifax Orly 94310 – France Tél. : +33 (L.G.) / Fax :

www.sazotech-solutions.fr// info@sazotech.fr

Collection teachers :

```
bdref> db.teachers.insertMany([
|   { _id: "Ens01", nom: "Dupont", prenom: "Marie", tel: null },
|   { _id: "Ens02", nom: "Durant", prenom: "Barbara", tel: "0600210111" },
|   { _id: "Ens03", nom: "Dupont", prenom: "Titi", tel: null }
| ])
|
{
  acknowledged: true,
  insertedIds: { '0': 'Ens01', '1': 'Ens02', '2': 'Ens03' }
}
bdref>
```

Collection courses

```
bdref> db.courses.insertMany([
|   { _id: "Info-01", intitule: "Bases de données", discipline: "Informatique", nbh: 30, resp: "Ens01" },
|   { _id: "Maths-01", intitule: "Mathématiques", discipline: "Mathématiques", nbh: 40, resp: "Ens02" },
|   { _id: "Info-02", intitule: "Complexité", discipline: "Informatique", nbh: 30, resp: "Ens01" }
| ])
|
{
  acknowledged: true,
  insertedIds: { '0': 'Info-01', '1': 'Maths-01', '2': 'Info-02' }
}
bdref>
```

Collection inscriptions :

```
bdref> db.inscriptions.insertMany([
|   { etudiant: "001", cours: "Info-01" },
|   { etudiant: "001", cours: "Maths-01" },
|   { etudiant: "001", cours: "Info-02" },
|   { etudiant: "002", cours: "Maths-01" },
|   { etudiant: "003", cours: "Info-01" },
|   { etudiant: "003", cours: "Info-02" }
| ])
|
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a5f61aea68c763b1903e4fc'),
    '1': ObjectId('6a5f61aea68c763b1903e4fd'),
    '2': ObjectId('6a5f61aea68c763b1903e4fe'),
    '3': ObjectId('6a5f61aea68c763b1903e4ff'),
    '4': ObjectId('6a5f61aea68c763b1903e500'),
    '5': ObjectId('6a5f61aea68c763b1903e501')
  }
}
bdref>
```

Approche B :

Approche bdimb (documents imbriqués)

Adresse 61 rue sifax Orly 94310 – France Tél. : +33 (L.G.) / Fax :

www.sazotech-solutions.fr// info@sazotech.fr

```
bdref> use bdimb
switched to db bdimb
bdimb>

bdimb>
```

Collection courses :

```
bdimb> db.courses.insertMany([
|   {
|     _id: "Info-01", intitule: "Bases de données", discipline: "Informatique", nbh: 30,
|     responsable: { code: "Ens01", nom: "Dupont", prenom: "Marie" }
|   },
|   {
|     _id: "Maths-01", intitule: "Mathématiques", discipline: "Mathématiques", nbh: 40,
|     responsable: { code: "Ens02", nom: "Durant", prenom: "Barbara" }
|   },
|   {
|     _id: "Info-02", intitule: "Complexité", discipline: "Informatique", nbh: 30,
|     responsable: { code: "Ens01", nom: "Dupont", prenom: "Marie" }
|   }
| ])
|
{
  acknowledged: true,
  insertedIds: { '0': 'Info-01', '1': 'Maths-01', '2': 'Info-02' }
}
bdimb>
```

Collection students (avec les cours imbriqués) :

Adresse 61 rue sifax Orly 94310 – France Tél. : +33 (L.G.) / Fax :

www.sazotech-solutions.fr// info@sazotech.fr

```
bdimb> db.students.insertMany([
|   {
|     _id: "001", nom: "Dupont", prenom: "Bob", tel: "0600210000",
|     cours: [
|       { code: "Info-01", intitule: "Bases de données" },
|       { code: "Maths-01", intitule: "Mathématiques" },
|       { code: "Info-02", intitule: "Complexité" }
|     ]
|   },
|   {
|     _id: "002", nom: "Durant", prenom: "Paul", tel: null,
|     cours: [ { code: "Maths-01", intitule: "Mathématiques" } ]
|   },
|   {
|     _id: "003", nom: "Dupont", prenom: "Toto", tel: null,
|     cours: [
|       { code: "Info-01", intitule: "Bases de données" },
|       { code: "Info-02", intitule: "Complexité" }
|     ]
|   }
| ])
|
{
  acknowledged: true,
  insertedIds: { '0': '001', '1': '002', '2': '003' }
}
bdimb>
```

2. Donner les avantages et inconvénients de chacune des approches proposées

Approche bdref (par références)

Avantages :

- Pas de duplication de données (le nom d'un enseignant n'est écrit qu'une fois, dans teachers)
- Facile à maintenir : si Marie Dupont change de téléphone, on modifie un seul document
- Bien adaptée aux relations many-to-many (un étudiant a plusieurs cours, un cours a plusieurs étudiants) grâce à la collection inscriptions
- Cohérence des données garantie (une seule source de vérité)

Inconvénients :

- MongoDB n'a pas de vraie jointure native comme en SQL : il faut utiliser $lookup dans une agrégation, qui est plus lourd en performance qu'un accès direct
- Nécessite plusieurs requêtes (ou un pipeline d'agrégation) pour reconstituer une vue complète (ex : un étudiant + tous ses cours + les infos du prof)

Approche bdimb (documents imbriqués)

Adresse 61 rue sifax Orly 94310 – France Tél. : +33 (L.G.) / Fax :

www.sazotech-solutions.fr// info@sazotech.fr

Avantages :

- Très rapide en lecture : une seule requête find() suffit pour récupérer un étudiant avec tous ses cours, ou un cours avec son responsable — pas de jointure nécessaire
- Modèle qui correspond bien à la façon dont on va souvent *afficher* les données (ex : la fiche complète d'un étudiant)

Inconvénients :

- Duplication des données : le nom de l'enseignant responsable est répété dans chaque cours qu'il dirige (Ens01 apparaît dans Info-01 et Info-02)
- Risque d'incohérence : si Marie Dupont change de nom, il faut penser à mettre à jour tous les documents où elle apparaît
- Les tableaux imbriqués (comme la liste des cours d'un étudiant) peuvent grossir sans limite si un cours a énormément d'étudiants — MongoDB a une limite de 16 Mo par document
- Moins flexible si on veut interroger les données autrement que prévu au départ (ex : "tous les étudiants d'un cours donné" devient moins direct)

3. Rédiger les requêtes suivantes pour chacune des approches proposées :
(a) Liste des cours disponibles

use bdref :

db.courses.find()

```
bdimb> use bdref
switched to db bdref
bdref>

bdref> db.courses.find()
[
  {
    _id: 'Info-01',
    intitule: 'Bases de données',
    discipline: 'Informatique',
    nbh: 30,
    resp: 'Ens01'
  },
  {
    _id: 'Maths-01',
    intitule: 'Mathématiques',
    discipline: 'Mathématiques',
    nbh: 40,
    resp: 'Ens02'
  },
  {
    _id: 'Info-02',
    intitule: 'Complexité',
    discipline: 'Informatique',
    nbh: 30,
    resp: 'Ens01'
  }
]
bdref>
```


Adresse 61 rue sifax Orly 94310 – France Tél. : +33 (L.G.) / Fax :

www.sazotech-solutions.fr// info@sazotech.fr

dans use bdimb :

```
bdref> use bdimb
switched to db bdimb
bdimb> db.courses.find()
[
  {
    _id: 'Info-01',
    intitule: 'Bases de données',
    discipline: 'Informatique',
    nbh: 30,
    responsable: { code: 'Ens01', nom: 'Dupont', prenom: 'Marie' }
  },
  {
    _id: 'Maths-01',
    intitule: 'Mathématiques',
    discipline: 'Mathématiques',
    nbh: 40,
    responsable: { code: 'Ens02', nom: 'Durant', prenom: 'Barbara' }
  },
  {
    _id: 'Info-02',
    intitule: 'Complexité',
    discipline: 'Informatique',
    nbh: 30,
    responsable: { code: 'Ens01', nom: 'Dupont', prenom: 'Marie' }
  }
]
bdimb>
```

(b) Liste des étudiants :

```
bdimb> use bdref
switched to db bdref
bdref> db.students.find()
[
  { _id: '001', nom: 'Dupont', prenom: 'Bob', tel: '0600210000' },
  { _id: '002', nom: 'Durant', prenom: 'Paul', tel: null },
  { _id: '003', nom: 'Dupont', prenom: 'Toto', tel: null }
]
bdref>
```


Adresse 61 rue sifax Orly 94310 – France Tél. : +33 (L.G.) / Fax :

www.sazotech-solutions.fr// info@sazotech.fr

```
bdimb> db.students.find()
[
  {
    _id: '001',
    nom: 'Dupont',
    prenom: 'Bob',
    tel: '0600210000',
    cours: [
      { code: 'Info-01', intitule: 'Bases de données' },
      { code: 'Maths-01', intitule: 'Mathématiques' },
      { code: 'Info-02', intitule: 'Complexité' }
    ]
  },
  {
    _id: '002',
    nom: 'Durant',
    prenom: 'Paul',
    tel: null,
    cours: [ { code: 'Maths-01', intitule: 'Mathématiques' } ]
  },
  {
    _id: '003',
    nom: 'Dupont',
    prenom: 'Toto',
    tel: null,
    cours: [
      { code: 'Info-01', intitule: 'Bases de données' },
      { code: 'Info-02', intitule: 'Complexité' }
    ]
  }
]
bdimb>
```

(c) Liste des enseignants

```
bdref> db.teachers.find()
[
  { _id: 'Ens01', nom: 'Dupont', prenom: 'Marie', tel: null },
  { _id: 'Ens02', nom: 'Durant', prenom: 'Barbara', tel: '0600210111' },
  { _id: 'Ens03', nom: 'Dupont', prenom: 'Titi', tel: null }
]
```

Sur bdimb : db.teachers.find() ne marche pas parcequ'il n'y a pas de collection teachers dans cette approche (le nom de l'enseignant est imbriqué dans chaque cours). On reconstitue donc la liste des enseignants à partir de courses, en dédupliquant :

db.courses.distinct("responsable")

```
bdimb> db.teachers.find()

bdimb> db.courses.distinct("responsable")
[
  { code: 'Ens01', nom: 'Dupont', prenom: 'Marie' },
  { code: 'Ens02', nom: 'Durant', prenom: 'Barbara' }
]
bdimb>
```


Adresse 61 rue sifax Orly 94310 – France Tél. : +33 (L.G.) / Fax :

www.sazotech-solutions.fr// info@sazotech.fr

(d) Liste des étudiants inscrits dans un cours d'informatique

bdref **: il faut passer par la collection inscriptions et faire une jointure ($lookup) vers courses pour filtrer sur la discipline, puis vers students pour récupérer les noms :**

```
bdref> db.inscriptions.aggregate([
|   { $lookup: { from: "courses", localField: "cours", foreignField: "_id", as: "c" } },
|   { $unwind: "$c" },
|   { $match: { "c.discipline": "Informatique" } },
|   { $lookup: { from: "students", localField: "etudiant", foreignField: "_id", as: "e" } },
|   { $unwind: "$e" },
|   { $project: { _id: 0, nom: "$e.nom", prenom: "$e.prenom" } }
| ])
|
[
  { nom: 'Dupont', prenom: 'Bob' },
  { nom: 'Dupont', prenom: 'Bob' },
  { nom: 'Dupont', prenom: 'Toto' },
  { nom: 'Dupont', prenom: 'Toto' }
]
bdref>
```

bdimb **: comme le champ discipline n'est pas dupliqué dans le document students (seulement code et intitule du cours), le plus direct est de filtrer sur les codes de cours d'informatique qu'on connaît (Info-01, Info-02) :**

```
bdimb> db.students.find(
|   { "cours.code": { $in: ["Info-01", "Info-02"] } },
|   { nom: 1, prenom: 1, _id: 0 }
| )
|
[ { nom: 'Dupont', prenom: 'Bob' }, { nom: 'Dupont', prenom: 'Toto' } ]
bdimb>
```

Inconvénient du modèle imbriqué : On n'a pas la discipline directement disponible côté étudiant, donc soit on la duplique aussi dans le tableau cours, soit on doit connaître les codes à l'avance.)

(e) Liste des cours suivis par l'étudiant Dupont Toto

bdref **: dans bdref, on part de inscriptions, on filtre sur l'étudiant "003", puis on fait une jointure vers courses :**

Adresse 61 rue sifax Orly 94310 – France Tél. : +33 (L.G.) / Fax :

www.sazotech-solutions.fr// info@sazotech.fr

```
bdimb> use bdref
switched to db bdref
bdref> db.inscriptions.aggregate([
|   { $match: { etudiant: "003" } },
|   { $lookup: { from: "courses", localField: "cours", foreignField: "_id", as: "c" } },
|   { $unwind: "$c" },
|   { $replaceRoot: { newRoot: "$c" } }
| ])
|
[
  {
    _id: 'Info-01',
    intitule: 'Bases de données',
    discipline: 'Informatique',
    nbh: 30,
    resp: 'Ens01'
  },
  {
    _id: 'Info-02',
    intitule: 'Complexité',
    discipline: 'Informatique',
    nbh: 30,
    resp: 'Ens01'
  }
]
bdref>
```

$replaceRoot sert à remplacer tout le document résultat par le sous-document c (le cours lui-même), pour avoir un affichage propre juste avec les infos du cours.

***bdimb* : Tout est déjà dans le document étudiant, pas besoin d'agrégation :**

```
bdref> use bdimb
switched to db bdimb
bdimb> db.students.findOne(
|   { nom: "Dupont", prenom: "Toto" },
|   { cours: 1, _id: 0 }
| )
|
{
  cours: [
    { code: 'Info-01', intitule: 'Bases de données' },
    { code: 'Info-02', intitule: 'Complexité' }
  ]
}
bdimb>
```

C'est exactement l'exemple qui illustre l'avantage du modèle imbriqué : une seule requête simple contre un pipeline d'agrégation dans l'autre approche.

(f) Liste des cours enseignés par Durand Barbara

Adresse 61 rue sifax Orly 94310 – France Tél. : +33 (L.G.) / Fax :

www.sazotech-solutions.fr// info@sazotech.fr

***bdref* : on part de courses, on fait une jointure vers teachers, puis on filtre sur le nom/prénom :**

```
bdimb> use bdref
switched to db bdref
bdref> db.courses.aggregate([
|   { $lookup: { from: "teachers", localField: "resp", foreignField: "_id", as: "t" } },
|   { $unwind: "$t" },
|   { $match: { "t.nom": "Durant", "t.prenom": "Barbara" } }
| ])
|
[
  {
    _id: 'Maths-01',
    intitule: 'Mathématiques',
    discipline: 'Mathématiques',
    nbh: 40,
    resp: 'Ens02',
    t: { _id: 'Ens02', nom: 'Durant', prenom: 'Barbara', tel: '0600210111' }
  }
]
bdref>
```

***bdimb* : comme le nom et prénom du responsable sont déjà imbriqués directement dans courses, on filtre directement, sans jointure :**

```
bdref> use bdimb
switched to db bdimb
bdimb> db.courses.find({
|   "responsable.nom": "Durant",
|   "responsable.prenom": "Barbara"
| })
|
[
  {
    _id: 'Maths-01',
    intitule: 'Mathématiques',
    discipline: 'Mathématiques',
    nbh: 40,
    responsable: { code: 'Ens02', nom: 'Durant', prenom: 'Barbara' }
  }
]
bdimb>
```

Encore un bon exemple de l'avantage du modèle imbriqué pour ce type de requête : pas besoin de $lookup.

(g) Liste des cours suivis par l'étudiant Dupont Bob avec les noms et prénoms des enseignants responsables de ces cours

***bdref* : on part de inscriptions, on filtre sur l'étudiant "001", puis on enchaîne deux jointures : une vers courses (pour avoir le cours) et une vers teachers (pour avoir le responsable) :**

Adresse 61 rue sifax Orly 94310 – France Tél. : +33 (L.G.) / Fax :

www.sazotech-solutions.fr// info@sazotech.fr

```
bdimb> use bdref
switched to db bdref
bdref> db.inscriptions.aggregate([
|   { $match: { etudiant: "001" } },
|   { $lookup: { from: "courses", localField: "cours", foreignField: "_id", as: "c" } },
|   { $unwind: "$c" },
|   { $lookup: { from: "teachers", localField: "c.resp", foreignField: "_id", as: "ens" } },
|   { $unwind: "$ens" },
|   { $project: { _id: 0, cours: "$c.intitule", enseignant_nom: "$ens.nom", enseignant_prenom: "$ens.prenom" } }
| ])
|
[
  {
    cours: 'Bases de données',
    enseignant_nom: 'Dupont',
    enseignant_prenom: 'Marie'
  },
  {
    cours: 'Mathématiques',
    enseignant_nom: 'Durant',
    enseignant_prenom: 'Barbara'
  },
  {
    cours: 'Complexité',
    enseignant_nom: 'Dupont',
    enseignant_prenom: 'Marie'
  }
]
bdref>
```

***bdimb* : le document étudiant contient les cours, mais seulement code et intitule (pas le responsable), donc il faut quand même une petite jointure vers courses pour récupérer le nom du prof imbriqué**

```
bdimb> db.students.aggregate([
|   { $match: { nom: "Dupont", prenom: "Bob" } },
|   { $unwind: "$cours" },
|   { $lookup: { from: "courses", localField: "cours.code", foreignField: "_id", as: "c" } },
|   { $unwind: "$c" },
|   { $project: { _id: 0, cours: "$c.intitule", enseignant_nom: "$c.responsable.nom", enseignant_prenom: "$c.responsable.prenom" }
}
| ])
|
[
  {
    cours: 'Bases de données',
    enseignant_nom: 'Dupont',
    enseignant_prenom: 'Marie'
  },
  {
    cours: 'Mathématiques',
    enseignant_nom: 'Durant',
    enseignant_prenom: 'Barbara'
  },
  {
    cours: 'Complexité',
    enseignant_nom: 'Dupont',
    enseignant_prenom: 'Marie'
  }
]
bdimb>
```

Même si le modèle est imbriqué , il a besoin d'un $lookup, parce que l'info du responsable n'est pas dupliquée dans le document étudiant , seulement dans courses. Ça illustre que le choix d'imbrication dépend vraiment des requêtes qu'on veut privilégier.)

Adresse 61 rue sifax Orly 94310 – France Tél. : +33 (L.G.) / Fax :

www.sazotech-solutions.fr// info@sazotech.fr